

DSA Revision Notes

Topic 3 — Stack, Queue & Monotonic Structures

Full theory · All operations · Internal implementation
Monotonic Stack · Deque · 15+ C++ examples · LeetCode hard problems

11

Sections

25+

C++ Examples

8

Visualizations

20+

LeetCode Problems

TOC Table of Contents

1. Stack — Theory, Memory Layout & STL

- LIFO principle · push/pop/top · array & linked-list implementation
- STL `std::stack` — all operations & complexity

2. Stack — Classic Problems

- Valid Parentheses · Min Stack · Evaluate RPN · Daily Temperatures
- Largest Rectangle in Histogram · Next Greater Element

3. Queue — Theory & STL

- FIFO principle · enqueue/dequeue · circular buffer implementation
- `std::queue` · `std::deque` — all operations

4. Queue — Classic Problems

- Implement Queue using Stacks · Sliding Window Maximum
- BFS using Queue · Design Circular Queue

5. Priority Queue (Heap)

- Min-heap / max-heap theory · `std::priority_queue`
- Top-K, K Closest Points, Task Scheduler

6. Monotonic Stack — Core Theory

- Increasing vs Decreasing · NGE / NSE template
- Daily Temperatures · Car Fleet · Sum of Subarray Minimums

7. Monotonic Stack — Advanced

- Largest Rectangle in Histogram · Maximal Rectangle (2D)
- Trapping Rain Water (stack approach) · Remove K Digits

8. Monotonic Deque

- Sliding Window Maximum · Shortest Subarray with Sum $\geq K$
- Constrained Subsequence Sum

9. Stack + DP Hybrid

- 132 Pattern · Remove Duplicate Letters · Decode String

10. Implement Everything from Scratch (C++)

- Array stack · Linked-list queue · Min-stack · Monotonic stack class

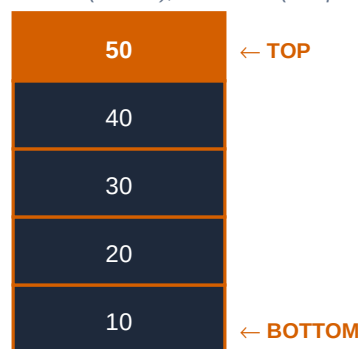
11. Complexity Cheat Sheet & Pattern Decision Tree

1 Stack — Theory, Memory Layout & STL

1.1 What is a Stack?

A **Stack** is a linear data structure that follows the **Last-In First-Out (LIFO)** principle. The element inserted last is the first one to be removed. Think of a stack of plates: you always add to the top and remove from the top. Real-world uses: function call stack, undo/redo, browser history, expression evaluation, DFS.

Stack: 10 was pushed first (bottom), 50 is TOP (last pushed)



■ Stack Operations — All $O(1)$

`push(x)` — Insert `x` onto the top of the stack. $\rightarrow O(1)$
`pop()` — Remove and return the top element. $\rightarrow O(1)$
`top()` — View the top element without removing. $\rightarrow O(1)$
`empty()` — Return true if stack has no elements. $\rightarrow O(1)$
`size()` — Return number of elements in stack. $\rightarrow O(1)$

1.2 Stack Implementation Using Array (from scratch)

The simplest stack implementation uses an array with a top-of-stack index. When `top == -1`, the stack is empty. When `top == capacity-1`, the stack is full (static). Dynamic arrays (vector) give us automatic resizing.

C++

```
1  #include <iostream>
2  #include <stdexcept>
3  using namespace std;
4
5  class ArrayStack {
6      int* arr;
7      int topIdx;
8      int capacity;
9  public:
10     ArrayStack(int cap = 1000) : capacity(cap), topIdx(-1) {
11         arr = new int[capacity];
12     }
13     ~ArrayStack() { delete[] arr; }
14
15     void push(int val) {
16         if (topIdx == capacity - 1) throw overflow_error("Stack Full");
17         arr[++topIdx] = val;        // pre-increment then assign
18     }
19     int pop() {
20         if (isEmpty()) throw underflow_error("Stack Empty");
21         return arr[topIdx--];      // return then post-decrement
22     }
23     int top() {
24         if (isEmpty()) throw underflow_error("Stack Empty");
25         return arr[topIdx];
26     }
27     bool isEmpty() { return topIdx == -1; }
28     int size()      { return topIdx + 1; }
29 };
30 // Time: All operations O(1)    Space: O(capacity)
```

1.3 Stack Implementation Using Linked List

Using a singly linked list: the HEAD acts as the TOP. push = insert at head ($O(1)$). pop = remove head ($O(1)$). No size limit (dynamic memory). Slightly higher constant due to node allocation.

C++

```
1  struct Node {
2      int val;
3      Node* next;
4      Node(int v) : val(v), next(nullptr) {}
5  };
6
7  class LinkedStack {
8      Node* head;    // head = TOP of stack
9      int _size;
10 public:
11     LinkedStack() : head(nullptr), _size(0) {}
12     ~LinkedStack() { while (!isEmpty()) pop(); }
13
14     void push(int val) {
15         Node* node = new Node(val);
16         node->next = head;    // new node points to old top
17         head = node;        // head is now new top
18         _size++;
19     }
20     int pop() {
21         if (isEmpty()) throw underflow_error("Stack Empty");
22         Node* tmp = head;
23         int val = tmp->val;
24         head = head->next;
25         delete tmp;
26         _size--;
27         return val;
28     }
29     int top()      { return head->val; }
30     bool isEmpty() { return head == nullptr; }
31     int size()     { return _size; }
32 };
33 // Time: All O(1)   Space: O(n) with pointer overhead per node
```

1.4 STL std::stack — Full Reference

C++

[illegible]

Operation	Method	Time	Notes
Push	st.push(x)	O(1)	Adds to top
Pop	st.pop()	O(1)	Removes top; returns void
Peek/Top	st.top()	O(1)	Returns reference to top element
Size	st.size()	O(1)	Number of elements
Empty check	st.empty()	O(1)	Returns bool
Emplace	st.emplace(x)	O(1)	Constructs in-place; faster for complex types
Swap	st.swap(other)	O(1)	Constant time — just swaps internal pointers

Clear	No direct method	O(n)	Pop in a loop or use destructor trick
-------	------------------	------	---------------------------------------

Table 1: `std::stack` operations with complexity

2 Stack — Classic Problems

2.1 Valid Parentheses (LC 20) — Easy

Check if a string of brackets is valid: every open bracket must be closed by the same type in the correct order. **Key idea:** push opening brackets; when closing bracket arrives, check if the stack top matches. If not, or stack is empty → invalid.

```
C++
1  #include <stack>
2  #include <unordered_map>
3  using namespace std;
4
5  bool isValid(string s) {
6      stack<char> st;
7      unordered_map<char, char> match = {{'}', '('}, {'}', '['}, {'}', '{'}}, {'}', '{'}};
8      for (char c : s) {
9          if (c=='(' || c=='[' || c=='{') {
10             st.push(c);           // push all opening brackets
11         } else {
12             // closing bracket: stack must be non-empty AND top must match
13             if (st.empty() || st.top() != match[c]) return false;
14             st.pop();
15         }
16     }
17     return st.empty(); // stack empty = all matched
18 }
19 // Time: O(n)   Space: O(n)
20
21 // Test cases:
22 // "()[]{}" → true    "([)]" → false
23 // "{[]}" → true     "]" → false
```

2.2 Min Stack (LC 155) — Medium

Design a stack that supports push, pop, top, and **getMin** in **O(1)**. Trick: maintain a parallel 'min stack' that tracks the current minimum at each level. When we push x, also push min(x, minSt.top()) onto minSt.

C++

```
1  class MinStack {
2      stack<int> st;
3      stack<int> minSt;    // parallel: minSt.top() = current min
4  public:
5      void push(int val) {
6          st.push(val);
7          // current min is either val or the previous min
8          int curMin = minSt.empty() ? val : min(val, minSt.top());
9          minSt.push(curMin);
10     }
11     void pop() {
12         st.pop();
13         minSt.pop();    // always pop both in sync
14     }
15     int top()    { return st.top(); }
16     int getMin() { return minSt.top(); } // O(1) always
17 };
18 // Time: All O(1)   Space: O(n) — two stacks
19
20 // Alternative: store (val, currentMin) pairs in one stack
21 // class MinStack {
22 //     stack<pair<int,int>> st;    // {value, min_so_far}
23 // public:
24 //     void push(int val) {
25 //         int m = st.empty() ? val : min(val, st.top().second);
26 //         st.push({val, m});
27 //     }
28 //     int getMin() { return st.top().second; }
29 // };
```

2.3 Evaluate Reverse Polish Notation (LC 150) — Medium

Evaluate an expression in RPN (postfix). Numbers → push. Operator → pop two operands (b then a), apply operator, push result. No parentheses needed — RPN handles precedence inherently.

C++

```
1  int evalRPN(vector<string>& tokens) {
2      stack<long long> st;
3      for (const string& tok : tokens) {
4          if (tok == "+" || tok == "-" || tok == "*" || tok == "/") {
5              long long b = st.top(); st.pop(); // second operand
6              long long a = st.top(); st.pop(); // first operand
7              if (tok == "+") st.push(a + b);
8              else if (tok == "-") st.push(a - b);
9              else if (tok == "*") st.push(a * b);
10             else st.push(a / b); // truncate toward zero
11         } else {
12             st.push(stoll(tok)); // convert string to long long
13         }
14     }
15     return (int)st.top();
16 }
17 // Time: O(n)   Space: O(n)
18
19 // Example: ["2", "1", "+", "3", "*"] → ((2+1)*3) = 9
20 // Step by step: push 2, push 1, pop→(2+1)=3 push, push 3, pop→(3*3)=9
```

2.4 Daily Temperatures (LC 739) — Medium

For each day, find how many days to wait for a warmer temperature. **Monotonic decreasing stack** of indices: when a warmer day arrives, pop all colder days and calculate the gap.

C++

```
1  vector<int> dailyTemperatures(vector<int>& T) {
2      int n = T.size();
3      vector<int> res(n, 0);
4      stack<int> st; // stores INDICES; temperatures are decreasing
5      for (int i = 0; i < n; i++) {
6          // Current temperature is warmer than stack top → found answer
7          while (!st.empty() && T[i] > T[st.top()]) {
8              int idx = st.top(); st.pop();
9              res[idx] = i - idx; // days waited
10         }
11         st.push(i); // push current index
12     }
13     // Remaining indices in stack have no warmer day → stay 0
14     return res;
15 }
16 // Time: O(n) — each index pushed/popped at most once
17 // Space: O(n) in worst case (decreasing temperatures)
18
19 // Example: T=[73,74,75,71,69,72,76,73]
20 // Res:      [1, 1, 4, 2, 1, 1, 0, 0]
```

2.5 Largest Rectangle in Histogram (LC 84) — Hard

Find the area of the largest rectangle that fits in the histogram. **Key insight:** for each bar, the maximum rectangle with that bar's height extends left until a shorter bar and right until a shorter bar. Monotonic increasing stack lets us find these boundaries efficiently.

C++

```
1  int largestRectangleArea(vector<int>& heights) {
2      stack<int> st;           // indices; heights are increasing
3      int maxArea = 0;
4      heights.push_back(0);   // sentinel: forces all bars to be processed
5      int n = heights.size();
6      for (int i = 0; i < n; i++) {
7          while (!st.empty() && heights[i] < heights[st.top()]) {
8              int h = heights[st.top()]; st.pop();
9              // Width: from current bar to new stack top (exclusive)
10             int w = st.empty() ? i : (i - st.top() - 1);
11             maxArea = max(maxArea, h * w);
12         }
13         st.push(i);
14     }
15     heights.pop_back();      // restore original
16     return maxArea;
17 }
18 // Time: O(n)   Space: O(n)
19
20 // Example: heights = [2,1,5,6,2,3]
21 // Largest rectangle: 5*2 = 10 (bars at index 2,3 with height 5)
```

■ Histogram Key Points

Why push 0 as sentinel? Without it, bars remaining in the stack at the end won't be processed.

The 0-height sentinel bar triggers popping all remaining bars cleanly.

Width formula: when stack is empty after pop, width = i (bar extends to left boundary).

when stack has elements, width = i - st.top() - 1 (gap between boundaries).

2.6 Generate Parentheses (LC 22) — Medium

Generate all valid combinations of n pairs of parentheses using backtracking with a stack-like counter. Rule: can add '(' if open < n; can add ')' if close < open.

C++

```
1  vector<string> generateParenthesis(int n) {
2      vector<string> result;
3      string current = "";
4      function<void(int, int)> backtrack = [&](int open, int close) {
5          if ((int)current.size() == 2 * n) {
6              result.push_back(current);
7              return;
8          }
9          if (open < n) {                // can still add '('
10             current += '(';
11             backtrack(open + 1, close);
12             current.pop_back();
13         }
14         if (close < open) {            // can add ')' only if open > close
15             current += ')';
16             backtrack(open, close + 1);
17             current.pop_back();
18         }
19     };
20     backtrack(0, 0);
21     return result;
22 }
23 // Time: O(4^n / sqrt(n)) - Catalan number   Space: O(n)
```

2.7 Decode String (LC 394) — Medium

Decode strings like **3[a2[bc]]** → **abcbcabcbcabcbcb**. Use two stacks: one for counts, one for partial strings. On '[': push current state. On ']': pop and repeat.

C++

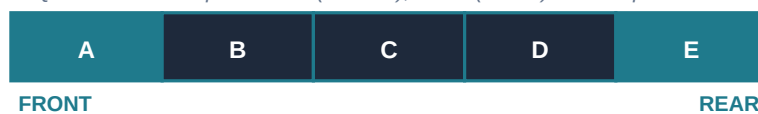
```
1  string decodeString(string s) {
2      stack<int>    countSt;    // stack of repeat counts
3      stack<string> strSt;      // stack of partial strings
4      string current = "";
5      int k = 0;
6      for (char c : s) {
7          if (isdigit(c)) {
8              k = k * 10 + (c - '0');    // handle multi-digit numbers
9          } else if (c == '[') {
10             countSt.push(k);            // save current count
11             strSt.push(current);        // save current string
12             current = "";               // reset for inner content
13             k = 0;
14         } else if (c == ']') {
15             int repeat = countSt.top(); countSt.pop();
16             string prev = strSt.top(); strSt.pop();
17             string expanded = "";
18             for (int i = 0; i < repeat; i++) expanded += current;
19             current = prev + expanded; // prepend saved string
20         } else {
21             current += c;
22         }
23     }
24     return current;
25 }
26 // Time: O(output length)   Space: O(nesting depth)
```

3 Queue — Theory & STL

3.1 What is a Queue?

A **Queue** is a linear data structure following the **First-In First-Out (FIFO)** principle. Elements are inserted at the **rear** (enqueue) and removed from the **front** (dequeue). Think of a line at a checkout: first person in is first served. Real-world uses: BFS, CPU task scheduling, I/O buffers, producer-consumer, printer queues.

Queue: A was enqueued first (FRONT), E last (REAR) — A dequeues next



■ Queue Operations — All $O(1)$

enqueue / push(x) — Insert x at the REAR. $\rightarrow O(1)$
dequeue / pop() — Remove element from the FRONT. $\rightarrow O(1)$
front() — View front element without removing. $\rightarrow O(1)$
back() — View rear element without removing. $\rightarrow O(1)$
empty() — Return true if queue is empty. $\rightarrow O(1)$
size() — Return number of elements. $\rightarrow O(1)$

3.2 Circular Queue Implementation (Array-based)

Using a fixed-size circular buffer avoids the $O(n)$ shift problem of a naive array queue. Two pointers: **head** (front) and **tail** (rear). Advance using modulo: $(tail + 1) \% \text{capacity}$.

C++

```
1  class MyCircularQueue {
2      vector<int> buf;
3      int head, tail, sz, cap;
4  public:
5      MyCircularQueue(int k) : buf(k), head(0), tail(-1), sz(0), cap(k) {}
6
7      bool enqueue(int val) {
8          if (isFull()) return false;
9          tail = (tail + 1) % cap;    // advance rear circularly
10         buf[tail] = val;
11         sz++;
12         return true;
13     }
14     bool dequeue() {
15         if (isEmpty()) return false;
16         head = (head + 1) % cap;    // advance front circularly
17         sz--;
18         return true;
19     }
20     int Front() { return isEmpty() ? -1 : buf[head]; }
21     int Rear() { return isEmpty() ? -1 : buf[tail]; }
22     bool isEmpty() { return sz == 0; }
23     bool isFull() { return sz == cap; }
24 };
25 // Time: All O(1)   Space: O(k)
```

3.3 STL std::queue & std::deque — Full Reference

C++

[illegible]

Operation	std::queue	std::deque	Time
Insert at rear	q.push(x)	dq.push_back(x)	O(1)
Insert at front	—	dq.push_front(x)	O(1)
Remove from front	q.pop()	dq.pop_front()	O(1)
Remove from rear	—	dq.pop_back()	O(1)
Access front	q.front()	dq.front()	O(1)
Access rear	q.back()	dq.back()	O(1)
Random access	—	dq[i]	O(1)
Insert at middle	—	dq.insert(it,x)	O(n)
Size / Empty	q.size() / empty	dq.size()/empty	O(1)

Table 2: `std::queue` vs `std::deque` operations

4 Queue — Classic Problems

4.1 Implement Queue using Two Stacks (LC 232) — Easy

Maintain two stacks: **inbox** (push) and **outbox** (pop). When outbox is empty and we need to pop/peek, dump all of inbox into outbox (reverses order). Each element moves at most twice → **amortized $O(1)$** per operation.

```
C++
1  class MyQueue {
2      stack<int> inbox;    // for push
3      stack<int> outbox;   // for pop/peek
4
5      void transfer() {    // move inbox → outbox
6          while (!inbox.empty()) {
7              outbox.push(inbox.top());
8              inbox.pop();
9          }
10     }
11 public:
12     void push(int x) {
13         inbox.push(x);    // always push to inbox
14     }
15     int pop() {
16         if (outbox.empty()) transfer();
17         int val = outbox.top();
18         outbox.pop();
19         return val;
20     }
21     int peek() {
22         if (outbox.empty()) transfer();
23         return outbox.top();
24     }
25     bool empty() { return inbox.empty() && outbox.empty(); }
26 };
27 // push: O(1) pop/peek: O(1) amortized (O(n) worst case transfer)
```

4.2 Implement Stack using Two Queues (LC 225) — Easy

After each push, rotate the queue so the new element is at the front. Every push is $O(n)$, but pop/top are $O(1)$.

C++

```
1  class MyStack {
2      queue<int> q;
3  public:
4      void push(int x) {
5          q.push(x);
6          // Rotate: move all elements before x to behind x
7          for (int i = 0; i < (int)q.size() - 1; i++) {
8              q.push(q.front());
9              q.pop();
10         }
11         // Now x is at the front - simulates LIFO top
12     }
13     int pop() { int v = q.front(); q.pop(); return v; }
14     int top() { return q.front(); }
15     bool empty() { return q.empty(); }
16 };
17 // push: O(n)  pop/top: O(1)
```

4.3 BFS using Queue — Template

Breadth-First Search explores all nodes at distance d before distance $d+1$. A queue naturally provides this level-by-level order. Mark visited before enqueueing to avoid duplicates.

5 Priority Queue (Heap)

5.1 Priority Queue Theory

A **Priority Queue** is an abstract data type where each element has a priority and the element with the highest (or lowest) priority is served first. Implemented using a **binary heap**: a complete binary tree satisfying the heap property.

Max-heap: parent $\geq$ children. **Min-heap**: parent $\leq$ children. C++ `std::priority_queue` is a max-heap by default.

■ Priority Queue Operations

`push(x)` — Insert x . Bubble up to restore heap property. $\rightarrow O(\log n)$

`pop()` — Remove top (max/min). Bubble down after swap. $\rightarrow O(\log n)$

`top()` — View max (or min) element. $\rightarrow O(1)$

`size()` — Number of elements. $\rightarrow O(1)$

`empty()` — True if heap is empty. $\rightarrow O(1)$

Building a heap from n elements (heapify): $\rightarrow O(n)$ [not $O(n \log n)$]

5.3 K Closest Points to Origin (LC 973) — Medium

C++

```
1  vector<vector<int>> kClosest(vector<vector<int>>& points, int k) {
2      // Max-heap by distance (keep k smallest → evict largest)
3      using P = pair<int, vector<int>>; // {dist_sq, point}
4      priority_queue<P> maxH;
5      for (auto& p : points) {
6          int dist = p[0]*p[0] + p[1]*p[1];
7          maxH.push({dist, p});
8          if ((int)maxH.size() > k)
9              maxH.pop(); // remove farthest
10     }
11     vector<vector<int>> res;
12     while (!maxH.empty()) {
13         res.push_back(maxH.top().second);
14         maxH.pop();
15     }
16     return res;
17 }
18 // Time: O(n log k)   Space: O(k)
```

6 Monotonic Stack — Core Theory

6.1 What is a Monotonic Stack?

A **monotonic stack** is a stack that maintains its elements in either **strictly increasing** or **strictly decreasing** order from bottom to top. When a new element would violate the order, we pop elements until the order is restored. This popping action tells us useful information about the relationship between elements — typically the **Next Greater Element** (NGE) or **Next Smaller Element** (NSE).

■ Two Types of Monotonic Stack

Monotonic DECREASING stack (bottom→top: large to small):

- Pop when new element is LARGER than top
- When we pop x because of new element y: y is the NEXT GREATER ELEMENT of x

Monotonic INCREASING stack (bottom→top: small to large):

- Pop when new element is SMALLER than top
- When we pop x because of new element y: y is the NEXT SMALLER ELEMENT of x

6.2 Next Greater Element (NGE) — Core Template

For each element, find the first element to its right that is greater than it. If none exists, answer is -1. This is the most fundamental monotonic stack pattern.

C++

```
1 // ■■ MONOTONIC DECREASING STACK – NGE Template ■■■■■■■■■■■■
2 vector<int> nextGreaterElement(vector<int>& nums) {
3     int n = nums.size();
4     vector<int> nge(n, -1); // default: no greater element
5     stack<int> st;          // stores INDICES (decreasing values)
6
7     for (int i = 0; i < n; i++) {
8         // nums[i] breaks the decreasing order → pop and record NGE
9         while (!st.empty() && nums[i] > nums[st.top()]) {
10             int idx = st.top(); st.pop();
11             nge[idx] = nums[i]; // nums[i] is NGE for nums[idx]
12         }
13         st.push(i);
14     }
15     return nge; // remaining in stack have nge = -1
16 }
17 // Time: O(n) – each element pushed/popped at most once
18 // Space: O(n)
19
20 // Example: nums = [2, 1, 5, 3, 6]
21 // NGE      = [5, 5, 6, 6, -1]
```

6.3 Next Greater Element I (LC 496) — Easy

C++

```
1 vector<int> nextGreaterElement(vector<int>& nums1, vector<int>& nums2) {
2     unordered_map<int, int> nge; // val → its next greater in nums2
3     stack<int> st;              // decreasing stack of values
4
5     for (int x : nums2) {
6         while (!st.empty() && x > st.top()) {
7             nge[st.top()] = x; // x is NGE for st.top()
8             st.pop();
9         }
10        st.push(x);
11    }
12    // Remaining in stack have no NGE → default to -1
13    vector<int> res;
14    for (int x : nums1)
15        res.push_back(nge.count(x) ? nge[x] : -1);
16    return res;
17 }
18 // Time: O(|nums1| + |nums2|)   Space: O(|nums2|)
```

6.4 Next Greater Element II — Circular Array (LC 503) — Medium

Array is circular: after the last element, we wrap back to the first. Trick: process the array **twice** (length $2n$) using index modulo n .

```
C++  
  
1  vector<int> nextGreaterElements(vector<int>& nums) {  
2      int n = nums.size();  
3      vector<int> res(n, -1);  
4      stack<int> st; // indices  
5  
6      for (int i = 0; i < 2 * n; i++) { // double pass for circularity  
7          int idx = i % n;  
8          while (!st.empty() && nums[idx] > nums[st.top()]) {  
9              res[st.top()] = nums[idx];  
10             st.pop();  
11         }  
12         if (i < n) st.push(idx); // only push in first pass  
13     }  
14     return res;  
15 }  
16 // Time: O(n)   Space: O(n)
```

6.5 Sum of Subarray Minimums (LC 907) — Medium

Find the sum of $\min(\text{subarray})$ for all subarrays. For each element, count how many subarrays it is the minimum of using **Previous Smaller Element** and **Next Smaller Element**. Count = $(i - \text{PSE}[i]) \times (\text{NSE}[i] - i)$. Use monotonic increasing stack.

C++

```
1  int sumSubarrayMins(vector<int>& arr) {
2      const int MOD = 1e9 + 7;
3      int n = arr.size();
4      long long ans = 0;
5      // Monotonic increasing stack stores {value, count}
6      // count = number of subarrays where this element extends as min
7      stack<pair<int,int>> st; // {arr[i], contribution_count}
8
9      for (int i = 0; i < n; i++) {
10         int cnt = 1; // at least the element itself
11         while (!st.empty() && st.top().first >= arr[i]) {
12             cnt += st.top().second; // absorb predecessor's count
13             ans -= (long long)st.top().first * st.top().second % MOD;
14             st.pop();
15         }
16         st.push({arr[i], cnt});
17         ans = (ans + (long long)arr[i] * cnt) % MOD;
18     }
19     return (int)ans;
20 }
21 // Time: O(n)   Space: O(n)
```

7 Monotonic Stack — Advanced Problems

7.1 Car Fleet (LC 853) — Medium

n cars travel to a target. Each car has a position and speed. A faster car behind a slower car cannot pass — they merge into a fleet. Sort cars by position (descending). Use a stack to track fleet times-to-target.

C++

```
1  int carFleet(int target, vector<int>& pos, vector<int>& speed) {
2      int n = pos.size();
3      vector<pair<int, double>> cars(n);
4      for (int i = 0; i < n; i++)
5          cars[i] = {pos[i], (double)(target - pos[i]) / speed[i]};
6
7      // Sort by position descending (closest to target first)
8      sort(cars.begin(), cars.end(), [](auto& a, auto& b){ return a.first > b.first; });
9
10     stack<double> st; // time for each fleet to reach target
11     for (auto& [p, t] : cars) {
12         st.push(t);
13         // If current car arrives <= car ahead, they merge (pop current)
14         if (st.size() >= 2) {
15             double cur = st.top(); st.pop();
16             if (cur <= st.top()) // merges into fleet ahead
17                 ; // discard cur (already popped)
18             else
19                 st.push(cur); // separate fleet
20         }
21     }
22     return (int)st.size();
23 }
24 // Time: O(n log n) sorting   Space: O(n)
```

7.2 Maximal Rectangle (LC 85) — Hard

Given a binary matrix, find the largest rectangle containing only 1s. Build a histogram row by row: $\text{heights}[j] =$ consecutive 1s ending at current row. Apply Largest Rectangle in Histogram (LC 84) on each row's histogram.

C++

```
1  int largestRectangleArea(vector<int>& h) {    // from LC 84
2      stack<int> st; int maxA = 0;
3      h.push_back(0);
4      for (int i = 0; i < (int)h.size(); i++) {
5          while (!st.empty() && h[i] < h[st.top()]) {
6              int height = h[st.top()]; st.pop();
7              int width = st.empty() ? i : i - st.top() - 1;
8              maxA = max(maxA, height * width);
9          }
10         st.push(i);
11     }
12     h.pop_back(); return maxA;
13 }
14
15 int maximalRectangle(vector<vector<char>>& matrix) {
16     if (matrix.empty()) return 0;
17     int rows = matrix.size(), cols = matrix[0].size();
18     vector<int> heights(cols, 0);
19     int maxArea = 0;
20     for (int r = 0; r < rows; r++) {
21         // Build histogram for this row
22         for (int c = 0; c < cols; c++) {
23             heights[c] = (matrix[r][c] == '1') ? heights[c] + 1 : 0;
24         }
25         maxArea = max(maxArea, largestRectangleArea(heights));
26     }
27     return maxArea;
28 }
29 // Time: O(rows * cols)    Space: O(cols)
```

7.3 Remove K Digits (LC 402) — Medium

Remove k digits from a number string to make it as small as possible. Use a **monotonic increasing stack**: when a new digit is smaller than the top, pop the top (this is one removal). Stop when we've removed k digits.

C++

```
1  string removeKdigits(string num, int k) {
2      string st = ""; // using string as a stack
3      for (char c : num) {
4          // Remove larger digits from top (greedy: smaller prefix = smaller number)
5          while (k > 0 && !st.empty() && st.back() > c) {
6              st.pop_back();
7              k--;
8          }
9          st += c;
10     }
11     // If k removals still remaining, remove from the end (largest suffix)
12     st.resize(st.size() - k);
13     // Remove leading zeros
14     int start = 0;
15     while (start < (int)st.size() - 1 && st[start] == '0') start++;
16     return st.substr(start);
17 }
18 // Time: O(n)   Space: O(n)
19
20 // Example: num="1432219", k=3 → "1219"
21 // Remove 4(>2→no wait), we go char by char:
22 // '1'→push '4'→push '3'→pop 4, push 3 '2'→pop 3, push 2 ...
```

7.4 Asteroid Collision (LC 735) — Medium

Positive asteroids move right, negative move left. Collision occurs when a right-moving (positive) meets a left-moving (negative). The smaller one explodes. If equal, both explode.

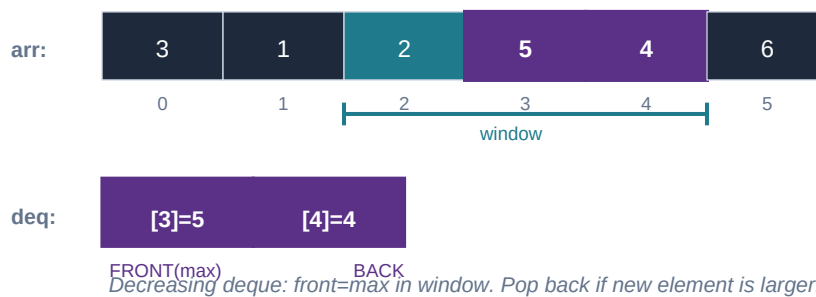
C++

```
1  vector<int> asteroidCollision(vector<int>& asteroids) {
2      stack<int> st;
3      for (int a : asteroids) {
4          bool exploded = false;
5          // Collision: top is positive (moving right), a is negative (moving left)
6          while (!st.empty() && a < 0 && st.top() > 0) {
7              if (st.top() < -a) {
8                  st.pop();          // stack asteroid explodes
9                  continue;         // current asteroid survives, keep checking
10             } else if (st.top() == -a) {
11                 st.pop();          // both explode
12             }
13             exploded = true;        // current asteroid exploded
14             break;
15         }
16         if (!exploded) st.push(a);
17     }
18     vector<int> res;
19     while (!st.empty()) { res.push_back(st.top()); st.pop(); }
20     reverse(res.begin(), res.end());
21     return res;
22 }
23 // Time: O(n)   Space: O(n)
```

8 Monotonic Deque

8.1 What is a Monotonic Deque?

A **monotonic deque** (double-ended queue) maintains elements in sorted order AND supports expiry from the front (elements leaving the window). It extends the monotonic stack concept to **sliding window** problems. The deque stores **indices** — values are implicitly ordered via the array.



■ Monotonic Deque Invariants

FRONT expiry: if $dq.front() \leq r - k$, it has left the window $\rightarrow pop_front()$

BACK insertion: before pushing index r , pop all back indices with smaller values $\rightarrow$ maintain decreasing order

RESULT: $dq.front()$ always holds the index of the MAXIMUM element in current window

For MINIMUM window: maintain INCREASING order instead (pop back if new element is smaller)

8.2 Sliding Window Maximum (LC 239) — Hard

C++

```
1  #include <deque>
2  using namespace std;
3
4  vector<int> maxSlidingWindow(vector<int>& nums, int k) {
5      deque<int> dq;          // stores INDICES; values are DECREASING
6      vector<int> result;
7      int n = nums.size();
8
9      for (int r = 0; r < n; r++) {
10         // Step 1: Remove expired index (out of window) from FRONT
11         while (!dq.empty() && dq.front() <= r - k)
12             dq.pop_front();
13
14         // Step 2: Maintain decreasing order – remove smaller from BACK
15         while (!dq.empty() && nums[dq.back()] <= nums[r])
16             dq.pop_back();
17
18         // Step 3: Push current index
19         dq.push_back(r);
20
21         // Step 4: Record max once window is fully formed
22         if (r >= k - 1)
23             result.push_back(nums[dq.front()]);
24     }
25     return result;
26 }
27 // Time: O(n) – each index pushed and popped at most once
28 // Space: O(k) – deque holds at most k indices
29
30 // nums=[1,3,-1,-3,5,3,6,7], k=3
31 // result=[3,3,5,5,6,7]
```

8.3 Shortest Subarray with Sum at Least K (LC 862) — Hard

This problem requires negative numbers support, so standard sliding window fails. Use prefix sums + monotonic deque. deque maintains increasing prefix sums (indices). For each r , pop front while $\text{prefix}[r] - \text{prefix}[\text{dq.front}()] \geq k$ (valid subarray found, minimize length).

C++

```
1  int shortestSubarray(vector<int>& nums, int k) {
2      int n = nums.size();
3      vector<long long> pre(n + 1, 0);
4      for (int i = 0; i < n; i++) pre[i+1] = pre[i] + nums[i];
5
6      deque<int> dq; // indices into pre[], values INCREASING
7      int res = INT_MAX;
8
9      for (int r = 0; r <= n; r++) {
10         // Try to form valid subarrays ending at r
11         while (!dq.empty() && pre[r] - pre[dq.front()] >= k) {
12             res = min(res, r - dq.front());
13             dq.pop_front(); // pop: we found best using this front
14         }
15         // Maintain increasing order of prefix sums
16         while (!dq.empty() && pre[dq.back()] >= pre[r])
17             dq.pop_back();
18         dq.push_back(r);
19     }
20     return res == INT_MAX ? -1 : res;
21 }
22 // Time: O(n)   Space: O(n)
```

9 Stack + DP Hybrid Problems

9.1 132 Pattern (LC 456) — Medium

Find if there exists $i < j < k$ such that $\text{nums}[j] < \text{nums}[k] < \text{nums}[i]$ (the '132' pattern). Traverse from right to left. Maintain a monotonic decreasing stack. Track 'third' = the largest value popped (this is the candidate for $\text{nums}[k]$). If current element $<$ third, pattern found.

C++

```
1  bool find132pattern(vector<int>& nums) {
2      stack<int> st;           // decreasing stack (candidates for nums[j])
3      int third = INT_MIN;    // best candidate for nums[k] (nums[k] < nums[j])
4
5      // Traverse RIGHT to LEFT
6      for (int i = nums.size() - 1; i >= 0; i--) {
7          // nums[i] < third means we found nums[i] < nums[k] < nums[j]
8          if (nums[i] < third) return true;
9
10         // Maintain decreasing stack; popped values are candidates for third
11         while (!st.empty() && nums[i] > st.top()) {
12             third = st.top(); // this becomes our best nums[k] candidate
13             st.pop();
14         }
15         st.push(nums[i]);
16     }
17     return false;
18 }
19 // Time: O(n)   Space: O(n)
```

9.2 Remove Duplicate Letters (LC 316) — Medium

Remove duplicates so each letter appears once and the result is the **smallest lexicographic order** among all such results. Greedy + monotonic stack: for each character, if it's already in the stack, skip. Otherwise, pop larger characters from the stack if they appear later in the string.

C++

```
1  string removeDuplicateLetters(string s) {
2      vector<int> lastIdx(26, 0); // last occurrence of each char
3      vector<bool> inStack(26, false);
4      stack<char> st;
5
6      for (int i = 0; i < (int)s.size(); i++)
7          lastIdx[s[i] - 'a'] = i;
8
9      for (int i = 0; i < (int)s.size(); i++) {
10         char c = s[i];
11         if (inStack[c - 'a']) continue; // already included
12
13         // Pop larger chars that will appear again later
14         while (!st.empty() && c < st.top() && lastIdx[st.top() - 'a'] > i) {
15             inStack[st.top() - 'a'] = false;
16             st.pop();
17         }
18         st.push(c);
19         inStack[c - 'a'] = true;
20     }
21     string res = "";
22     while (!st.empty()) { res += st.top(); st.pop(); }
23     reverse(res.begin(), res.end());
24     return res;
25 }
26 // Time: O(n)   Space: O(1) — bounded by 26 letters
```

9.3 Online Stock Span (LC 901) — Medium

For each day's price, find how many consecutive days (including today) the price was $\leq$ today's price. Use a stack of (price, span) pairs.

C++

```
1  class StockSpanner {
2      stack<pair<int,int>> st; // {price, span}
3  public:
4      int next(int price) {
5          int span = 1;
6          // Merge all days with price <= current into current's span
7          while (!st.empty() && st.top().first <= price) {
8              span += st.top().second;
9              st.pop();
10         }
11         st.push({price, span});
12         return span;
13     }
14 };
15 // Time: O(1) amortized per call   Space: O(n)
16
17 // Prices: [100, 80, 60, 70, 60, 75, 85]
18 // Spans:  [ 1,  1,  1,  2,  1,  4,  6]
```

10 Implement Everything from Scratch (C++)

10.1 Complete Monotonic Stack Class

C++

```
1 // Reusable MonotonicStack supporting NGE, NSE, and span queries
2 #include <vector>
3 #include <stack>
4 using namespace std;
5
6 class MonotonicStack {
7 public:
8     // Next Greater Element for each index - O(n)
9     static vector<int> nge(const vector<int>& arr) {
10         int n = arr.size();
11         vector<int> res(n, -1);
12         stack<int> st; // decreasing stack of indices
13         for (int i = 0; i < n; i++) {
14             while (!st.empty() && arr[i] > arr[st.top()]) {
15                 res[st.top()] = arr[i]; st.pop();
16             }
17             st.push(i);
18         }
19         return res;
20     }
21
22     // Previous Smaller Element for each index - O(n)
23     static vector<int> pse(const vector<int>& arr) {
24         int n = arr.size();
25         vector<int> res(n, -1);
26         stack<int> st; // increasing stack of indices
27         for (int i = 0; i < n; i++) {
28             while (!st.empty() && arr[st.top()] >= arr[i]) st.pop();
29             res[i] = st.empty() ? -1 : arr[st.top()];
30             st.push(i);
31         }
32         return res;
33     }
34
35     // Next Smaller Element for each index - O(n)
36     static vector<int> nse(const vector<int>& arr) {
37         int n = arr.size();
38         vector<int> res(n, -1);
39         stack<int> st; // increasing stack of indices
40         for (int i = 0; i < n; i++) {
41             while (!st.empty() && arr[i] < arr[st.top()]) {
42                 res[st.top()] = arr[i]; st.pop();
43             }
44             st.push(i);
45         }
46         return res;
47     }
48 };
```


11.3 Complete LeetCode Problem Map

#	Problem	Pattern	Diff
20	Valid Parentheses	Stack — bracket match	Easy
22	Generate Parentheses	Stack / Backtrack	Medium
84	Largest Rectangle in Histogram	Monotonic Inc. Stack	Hard
85	Maximal Rectangle	Histogram repeated	Hard
150	Evaluate Reverse Polish Notation	Stack — RPN eval	Medium
155	Min Stack	Paired Stack	Medium
225	Implement Stack using Queues	Two Queues	Easy
232	Implement Queue using Stacks	Two Stacks amortized	Easy
239	Sliding Window Maximum	Monotonic Deque	Hard
316	Remove Duplicate Letters	Mono Stack + greedy	Medium
394	Decode String	Two Stacks	Medium
402	Remove K Digits	Mono Inc Stack	Medium
456	132 Pattern	Mono Stack right-to-left	Medium
496	Next Greater Element I	Monotonic Dec Stack	Easy
503	Next Greater Element II	Mono Stack circular	Medium
735	Asteroid Collision	Stack simulation	Medium
739	Daily Temperatures	Monotonic Dec Stack	Medium
853	Car Fleet	Sort + Stack	Medium
862	Shortest Subarray Sum $\geq K$	Prefix + Mono Deque	Hard
901	Online Stock Span	Stack with span pairs	Medium
907	Sum of Subarray Minimums	Mono Inc Stack contribution	Medium
973	K Closest Points to Origin	Min-Heap / Priority Queue	Medium
215	Kth Largest Element	Min-Heap size K	Medium

Table 4: Complete LeetCode problem map for Stack, Queue & Monotonic Structures

■ Golden Rules — Stack, Queue & Monotonic Structures

1. Stack = LIFO. Queue = FIFO. Both core operations $O(1)$. Choose based on traversal order needed.
2. Monotonic stack: every element is pushed and popped at most once $\rightarrow O(n)$ despite the while loop.
3. When 'pop' reveals the answer: popping x because of y means y is NGE/NSE of x .
4. Monotonic deque = mono stack + front expiry. Use for sliding window max/min queries.
5. Min Stack: always maintain a parallel min-tracker. Never recompute — store state on push.
6. Histogram rectangle: sentinel 0 at end forces all remaining stack elements to be processed.
7. Circular array NGE: run the loop for $2n$ iterations using index $\% n$. Push only in first n .
8. Priority queue default = MAX-heap. For min-heap: use `greater<T>` as comparator.
9. Two stacks simulate a queue with amortized $O(1)$: each element crosses the transfer at most once.
10. Deque is the Swiss Army knife: $O(1)$ front/back push/pop AND $O(1)$ random access.